AUTOM-03: Monaco Configuration-as-Code

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 3 of 9 | **Created:** January 2026 | **Last Updated:** 10/29/2026

Monaco (Monitoring as Code) is Dynatrace's official CLI tool for configuration management. It uses YAML files to define configurations and supports version control, CI/CD integration, and multi-environment deployments.

Table of Contents

1. Introduction
2. Getting Started
3. Project Structure
4. Configuration Files
5. Common Operations
6. Advanced Features
7. Next Steps

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Monaco CLI	Installed from the GitHub releases binary (Dynatrace publishes no Homebrew formula)
Platform token or OAuth client	Always `app-engine:apps:run`, `settings:objects:read`, `settings:objects:write` and `settings:schemas:read` — once a platform credential is configured, Monaco sends Settings 2.0 through it. Add the scopes for each platform type you manage: SLOs need `slo:slos:read`, `slo:slos:write` and `slo:objective-templates:read`; workflows, documents, segments and buckets are listed on the access-handling page
API Token	`DataExport` (always), `settings.read` / `settings.write` (used for Settings 2.0 only when no platform credential is configured), plus `ReadConfig` / `WriteConfig` for classic Configuration APIs
Tenant URL	The **platform** URL, `https://<env-id>.apps.dynatrace.com`, whenever a platform token or OAuth client is used

> **Sources:** Monaco manifest and resources (DT docs) — `app-engine:apps:run`: *"This permission is required to access Dynatrace Platform metadata endpoints."*; Monaco API support and access handling (DT docs) — per-type scopes; client set (Dynatrace GitHub).
> **Derived:** Settings 2.0 through the platform credential is read from that file at v2.30.0, which builds the classic settings client only when no platform credential is configured.

Learning Objectives

By the end of this notebook, you will:

- Understand Monaco's configuration-as-code model
- Know how to download existing configurations
- Be able to create and deploy configurations via YAML
- Handle multi-environment deployments

---

1. Introduction

Why Monaco?

Benefit	Description
Version Control	Track all changes in Git
Repeatability	Deploy same config to multiple environments
Review Process	Use pull requests for config changes
Disaster Recovery	Quickly restore configurations
Documentation	YAML files serve as living documentation

Monaco vs Direct API

Aspect	Monaco	Settings API
Format	YAML files	JSON payloads
State management	Implicit — the config coordinate (`project` / `type` / `id`); classic APIs match by name	Manual tracking
Multi-tenant	Built-in support	Custom scripting
Dry run	`--dry-run` — structure and references only, no API calls	`validateOnly=true` — server-side validation of the payload, nothing saved
Delete	The entries listed in a `delete.yaml` file (`delete` command)	Manual

> **Sources:** Monaco configuration YAML reference (DT docs) — *"The name property isn't used to identify Dynatrace objects. Instead, the configuration's coordinate, a combination of project, type, and configuration ID, or originObjectId, if present, is used."*; Settings API — POST an object (DT docs) — *"the request runs only validation of the submitted settings objects, without saving them."*

> **Already a Terraform shop?** See **AUTOM-01 §5: When a Terraform Shop Should Add Monaco** for the four patterns where Monaco still earns its place — most commonly bulk download from an existing tenant, multi-tenant identical-config deployment, and app-team self-service without state management.

---

2. Getting Started

Installation

Monaco ships as a single binary on GitHub releases; there is no Homebrew formula. Pick the asset for your OS and architecture (the install guide also lists `monaco-darwin-amd64`, `monaco-linux-arm64`, `monaco-linux-386` and a SHA-256 file for each):

macOS (Apple silicon):

```
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-darwin-arm64 -o monaco
chmod +x monaco
```

Linux (x86-64):

```
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
chmod +x monaco
```

Windows: download the `.exe` asset from the same releases page.

Verify installation:

```
monaco version
```

Environment Setup

Create environment variables for authentication:

```
export DT_TENANT_URL="https://&lt;env-id&gt;.apps.dynatrace.com"   # platform URL, not .live.
export DT_API_TOKEN="&lt;your-api-token&gt;"
export DT_PLATFORM_TOKEN="&lt;your-platform-token&gt;"   # SLOs, workflows, documents, segments, buckets
```

With a platform token or OAuth client in play, the URL must be the **platform** (`.apps.`) URL: Monaco first asks it for the environment's classic URL (that lookup is what needs `app-engine:apps:run`) and sends access-token calls there itself. A `.live.` URL fails with *"make sure you are using a Dynatrace Platform URL and not a Dynatrace Classic URL when authenticating with an OAuth client or platform token"*. Only a run with an access token alone takes the classic URL directly.

Monaco never takes a secret on the command line. Both the manifest and the `--token` download flag take the **name** of the environment variable that holds the token (`DT_API_TOKEN`), not its value.

> **Sources:** metadata client (Dynatrace GitHub) — the error quoted above; Monaco manifest and resources (DT docs) — every manifest example uses `.apps.dynatrace.com`.

Your First Download

Download all configurations from your tenant:

```
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --platform-token DT_PLATFORM_TOKEN \
  --output-folder ./downloaded-config
```

`--token` and `--platform-token` name environment variables (no `$`). The access token covers the classic Configuration APIs; the platform token — or the `--oauth-client-id` / `--oauth-client-secret` pair — covers platform configurations (SLOs, workflows, documents, buckets, segments) and, once given, Settings 2.0 too. Access tokens and platform tokens are not interchangeable. Dropping the platform token limits the download to Settings 2.0 and classic configurations, and `--url` must then be the classic `.live.` URL.

Download only the platform configuration types — SLOs, segments, workflows and documents. The `--only-*` flags can be combined (Monaco 2.23.1+):

```
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --platform-token DT_PLATFORM_TOKEN \
  --output-folder ./downloaded-config \
  --only-slo-v2 --only-segments --only-automation --only-documents
```

The full set of `--only-*` flags is `--only-settings`, `--only-apis`, `--only-slo-v2`, `--only-segments`, `--only-automation`, `--only-documents`, `--only-buckets` and `--only-openpipeline` (read from the download command's source, `cmd/monaco/download/download_command.go`, at v2.30.0). Monaco 2.31.0 (10/23/2026) removed the deprecated `openpipeline` type — the release notes (Dynatrace GitHub): *"Removed support for deprecated OpenPipeline configurations (type `openpipeline`)"* — and its download command no longer has `--only-openpipeline`. OpenPipeline configuration is Settings 2.0 objects, so download it with `--only-settings` or `--settings-schema` with each `builtin:openpipeline.*` schema in use, plus `--admin-access` for pipelines other users own.

Settings owned by other users (Monaco CLI 2.30.0+). A download returns the settings objects the credential's user can see. For OpenPipeline, `--admin-access` also returns pipelines other users own — the release note: *"enabling this flag will also download configurations of different owners (other than the user that is assigned to the platform token or OAuth client). Requires `settings:objects:admin`."* — in addition to the `settings:objects:read` scope every download needs. The flag applies to owner-based OpenPipeline schemas only (`builtin:openpipeline.*`); on older CLI versions there is no equivalent. See Using a Download as a Backup for what else a download leaves out.

```
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --platform-token DT_PLATFORM_TOKEN \
  --output-folder ./downloaded-config \
  --only-settings --admin-access
```

> **Sources:** Monaco v2.30.0 release notes (Dynatrace GitHub) — *"If settings-based OpenPipeline configurations (`builtin:openpipeline.*`) are downloaded, enabling this flag will also download configurations of different owners"*; settings client (Dynatrace GitHub) — *"supportsAdminAccess returns true if the schema is an owner-based OpenPipeline schema."*

Classic — inventorying what you still have before the upgrade. Download specific Settings 2.0 schemas with `--settings-schema` (`--api` is for **classic** Configuration APIs only):

```
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --platform-token DT_PLATFORM_TOKEN \
  --output-folder ./downloaded-config \
  --settings-schema builtin:management-zones,builtin:tags.auto-tagging
```

> **Downloading classic schemas is the point here, not an oversight.** Both schemas above are marked **Blocked at upgrade** in AUTOM-02's Settings 2.0 catalog — they work today and stop answering once your tenant moves to the latest Dynatrace. That makes `monaco download` the fastest way to *inventory* what you still have before migrating it: every object, in version control, in one command. Keep the export as your migration worksheet; retarget the deployment. What it should not become is a template for new configuration.

---

3. Project Structure

Recommended Layout

```
dynatrace-config/
├── manifest.yaml             # Projects and target environments (environmentGroups)
└── projects/
    └── my-project/
        ├── slo-v2/               # service-level objectives (modern SLO app)
        │   └── config.yaml
        ├── segments/
        │   └── config.yaml
        ├── workflows/            # problem routing and other automations
        │   └── config.yaml
        └── anomaly-detectors/    # builtin:davis.anomaly-detectors (Settings 2.0)
            └── config.yaml
```

Classic estates add `management-zones/`, `auto-tagging/` and `alerting-profiles/` here until the upgrade — see the *Classic* blocks in §4.

Manifest File

The `manifest.yaml` defines your projects **and** your environments — Monaco v2 has no separate environment files:

```
manifestVersion: 1.0

projects:
  - name: my-project
    path: projects/my-project

environmentGroups:
  - name: default
    environments:
      - name: development
        url:
          type: environment
          value: DT_DEV_URL
        auth:
          token:
            name: DT_DEV_TOKEN
          platformToken:
            name: DT_DEV_PLATFORM_TOKEN
      - name: production
        url:
          type: environment
          value: DT_PROD_URL
        auth:
          token:
            name: DT_PROD_TOKEN
          platformToken:
            name: DT_PROD_PLATFORM_TOKEN
```

`url` accepts `type: environment` + `value: <VAR_NAME>`, but every `auth` entry takes only `name: <VAR_NAME>` — Monaco always loads secrets from environment variables. `platformToken` (Monaco 2.24.0+) — or an `oAuth` client in its place — is what lets the project manage SLOs, workflows, documents, buckets and segments; drop it only for a project that manages Settings 2.0 and classic configuration alone. With `platformToken` or `oAuth` set, each `url` variable holds the platform (`.apps.`) URL.

---

4. Configuration Files

Basic Configuration Structure

Every config is an `id`, a `config:` block (name, template, parameters) and a `type:`. For a Settings 2.0 object the type names a schema; for a platform object it names the platform type:

```
configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: slo.json
    type: slo-v2
```

SLO Example (`type: slo-v2`)

slo-v2/config.yaml:

```
configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: checkout-availability.json
    type: slo-v2
```

slo-v2/checkout-availability.json:

```
{
  "name": "{{ .name }}",
  "description": "Request success ratio, rolling 30 days",
  "tags": ["app:checkout", "owner:payments"],
  "criteria": [
    {
      "target": 99.5,
      "warning": 99.9,
      "timeframeFrom": "now-30d",
      "timeframeTo": "now"
    }
  ],
  "customSli": {
    "indicator": "timeseries { total = sum(dt.service.request.count), failures =
sum(dt.service.request.failure_count) }\n| fieldsAdd sli = ((total[] - failures[]) / total[]) * 100\n|
fieldsRemove total, failures"
  }
}
```

The payload is the SLO Service API's own shape — `criteria` is a **list**, and the SLI is a DQL `indicator` producing a timeseries field named `sli` (SLO-02's availability query, minus `from:` / `interval:`, which the criteria supply). It matches Monaco's own `slo-v2` integration-test template field for field. `type: slo-v2` needs Monaco **v2.22.0+** and platform auth, and despite the name it is the **modern** SLO app — the classic SLO is a different Monaco type, and the Terraform resource `dynatrace_slo_v2` is classic too (SLO-05).

Other Platform Types

Type block	Manages	Since
`type: slo-v2`	SLOs on the modern SLO app	v2.22.0
`type: segment`	Segments (the filtering successor to management zones)	v2.19.0
`type: {automation: {resource: workflow}}` — also `business-calendar`, `scheduling-rule`	Workflows (the routing successor to alerting profiles + problem notifications)	v2.6.0
`type: {document: {kind: dashboard}}` — also `notebook`, `launchpad`; optional `private`	Dashboards, notebooks, launchpads	v2.15.0 (launchpad v2.18.0)
`type: bucket`	Grail buckets	v2.9.0
`type: {settings: {schema: builtin:davis.anomaly-detectors, scope: environment}}`	Davis anomaly detectors (the successor to custom metric events) — a Settings 2.0 schema that carries forward	—

Download a workflow or segment before you author one. A workflow template is the full Automation API payload (trigger, tasks, positions) and a segment template is the segment editor's JSON filter tree — neither is sensible to

hand-write. Build one in the app, `monaco download --only-automation` / `--only-segments`, and parameterize the result.

> **Sources:** Monaco YAML configuration — type fields (DT docs) — the type blocks and version gates in the table; `slo-v2` test template (Dynatrace GitHub) — payload shape, read 09/28/2026; Monaco v2.22.0 release notes (Dynatrace GitHub) — *"Support for service-level objectives leveraging Grail"*.

Classic Configuration — existing estates on unupgraded tenants

> **Dynatrace Classic — maintain, don't author.** Management zones, auto-tagging and alerting profiles are the configurations most existing Monaco projects contain, so their shape is kept below. All three are marked **Blocked at upgrade** in AUTOM-02's Settings 2.0 catalog and are on Dynatrace's published Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs) — *"None of the schemas on this page are visible in Latest Dynatrace."*
>
> That matters more for Monaco than for the raw API. Monaco resolves a schema before writing objects, so once a schema is removed, `monaco deploy` fails for every config that references it — and it fails at deploy time, in your pipeline, not at edit time. Successors: management zone → `segment` (filtering) + IAM policies (access); alerting profile → `automation` workflow; auto-tagging → primary fields/tags set at ingest (FAQ-02).

Management Zone Example (classic)

config.yaml:

```
configs:
  - id: production-mz
    config:
      name: Production
      template: production-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment

  - id: staging-mz
    config:
      name: Staging
      template: staging-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment
```

production-mz.json:

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "conditions": [
        {
          "key": {
            "type": "STATIC",
            "attribute": "SERVICE_TAGS"
          },
          "comparisonInfo": {
            "type": "TAG",
            "operator": "EQUALS",
            "value": {
              "context": "CONTEXTLESS",
              "key": "environment",
              "value": "production"
            },
            "negate": false
          }
        }
      ]
    }
  ]
}
```

Auto-Tagging Example (classic)

config.yaml:

```
configs:
  - id: application-tag
    config:
      name: Application
      template: application-tag.json
    type:
      settings:
        schema: builtin:tags.auto-tagging
        scope: environment
```

application-tag.json:

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "valueFormat": "{Service:DetectedName}",
      "propagationTypes": ["SERVICE_TO_PROCESS_GROUP_LIKE"],
      "conditions": []
    }
  ]
}
```

Using Variables

Monaco v2 reads environment variables through `environment` **parameters**, declared under `parameters:` with a `name` (the variable) and an optional `default`. The template then refers to the parameter by its key — there is no `{{ .Env.X }}` syntax in v2:

config.yaml:

```
configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: checkout-availability.json
      parameters:
        owner_team:
          type: environment
          name: OWNER_TEAM
        slo_target:
          type: environment
          name: SLO_TARGET
          default: "99.5"
    type: slo-v2
```

In `checkout-availability.json`, use `"tags": ["owner:{{ .owner_team }}"]` and `"target": {{ .slo_target }}` — unquoted, so the rendered JSON carries a number. Without a `default`, the deployment fails if the variable is unset.

---

5. Common Operations

Deploy Configurations

Deploy to all environments:

```
monaco deploy manifest.yaml
```

Deploy to specific environment:

```
monaco deploy manifest.yaml --environment production
```

Dry run (check file structure without applying):

```
monaco deploy manifest.yaml --dry-run
```

Validate Configurations

Monaco does **not** ship a standalone `monaco validate` subcommand. Validation runs as part of `monaco deploy --dry-run`, which loads the manifest, parses every project, checks that templates are valid JSON, and resolves cross-references — **without contacting the tenant**. It cannot validate the JSON payload against the schema, and it does not check reachability or token scopes: a deploy can still fail with HTTP 400 after a clean dry-run.

```
monaco deploy manifest.yaml --dry-run
```

Delete Configurations

Remove the configurations listed in a delete file — `delete` does not look for orphans; it deletes exactly what the file names. The manifest is passed via `--manifest`, not as a positional argument, and there is **no** `--dry-run` for `delete`:

```
monaco delete --manifest manifest.yaml --file delete.yaml
```

delete.yaml:

```
delete:
  - project: my-project
    type: slo-v2
    id: checkout-availability
  # Classic, until the upgrade — a Settings 2.0 entry takes the schema as its type
  - project: my-project
    type: builtin:management-zones
    id: old-management-zone-id
```

Every non-classic entry needs `project` **and** `id` **together** (the Monaco config coordinate) or an `objectId` on its own; Monaco's delete-file loader rejects an entry with `id` and no `project`. The `type` is the config type (`slo-v2`, `segment`, `workflow`, `document`, `bucket`) or, for Settings 2.0, the schema ID.

> **Sources:** Monaco commands reference (DT docs) — *"As input, the delete command requires two YAML files: The manifest file that specifies the relevant Dynatrace environments. The deletefile that specifies the relevant configurations."*; delete command (Dynatrace GitHub) — flags `--manifest`, `--file`, `--group`, `--environment` only.

Using a Download as a Backup — What It Does Not Capture

`monaco download` is a good **configuration** backup: Settings 2.0 objects, workflows, business calendars and scheduling rules, bucket definitions, dashboards, notebooks, launchpads, segments and `slo-v2` SLOs all land in version control. It is not a **tenant** backup. Some things are filtered out by design, some are invisible to the credential you run it with, and some have no Monaco type at all. Keep the list below next to the export, so that a restore does not discover the gaps for you.

What is missing	Why	What to do
Credentials and secrets — `aws-credentials`, `azure-credentials`, `kubernetes-credentials`, `credential-vault`, `extension`	Excluded by default: *"Typically, these types contain secrets that must not be exported"*	Inventory every credential; recreate by hand on restore. Turning the filter off does not help — the API does not return the secret
Read-only settings	*"Read-only configurations are also excluded."*	Usually platform defaults you would not restore. `MONACO_FEAT_DOWNLOAD_FILTER_SETTINGS_UNMODIFIABLE=false` downloads them

What is missing	Why	What to do
Settings Monaco filters by default — notably `builtin:host.monitoring.mode`, skipped entirely, plus a handful of built-in defaults (the `Default` alerting profile, `default` bucket rules)	The source skips host monitoring mode because it *"is not reliable during download"*	If per-host monitoring modes matter to a restore, run a separate download with `MONACO_FEAT_DOWNLOAD_FILTER_SETTINGS=false`. Dynatrace warns that an unfiltered download needs *"some manual post-processing"* before it deploys
OpenPipeline owned by other users	Without `--admin-access` (Monaco CLI 2.30.0+), only the credential user's pipelines are returned	Add `--admin-access` with `settings:objects:admin`. Back up OpenPipeline through `settings` — the owner-based `builtin:openpipeline.*` schemas, which `--admin-access` downloads for all owners
Other settings owned by other users	Objects created with `allUsers: none` are visible only to their owner, and `--admin-access` covers OpenPipeline schemas only	In community practice, run the download as a dedicated service identity and have owners share objects with it — then compare object counts per schema against the UI before trusting the export
Documents	*"Monaco does not download Ready-made documents."* Documents the credential cannot see are not returned, and a public document the Monaco user does not own may fail to redeploy	Ready-made documents ship with the platform. For the rest, transfer ownership of shared documents to the backup identity, or accept that private documents stay with their owners

What is missing	Why	What to do
Account resources	Users, service users, groups and policies are a separate command; *"existing commands like `monaco deploy` ignore any account configuration"*	Run `monaco account download` alongside the environment download. Boundaries are included from Monaco 2.26.0 (*"Account management: deploy, download, and delete boundaries"*)
Things with no Monaco type	The type-fields page (read 09/29/2026) lists no type for lookup tables, installed Hub or custom apps, access tokens, platform tokens or OAuth clients	Record them in the runbook; reinstall apps and re-mint tokens and clients on restore
Anything that is not configuration	Grail data (a `bucket` config is the bucket's definition, not its contents), Davis baselines and problem history	Not recoverable from any configuration export — see FAQ-25 on accumulated state

A backup run that closes the gaps Monaco can close:

```
export MONACO_FEAT_DOWNLOAD_FILTER_SETTINGS_UNMODIFIABLE=false   # optional: include read-only
settings

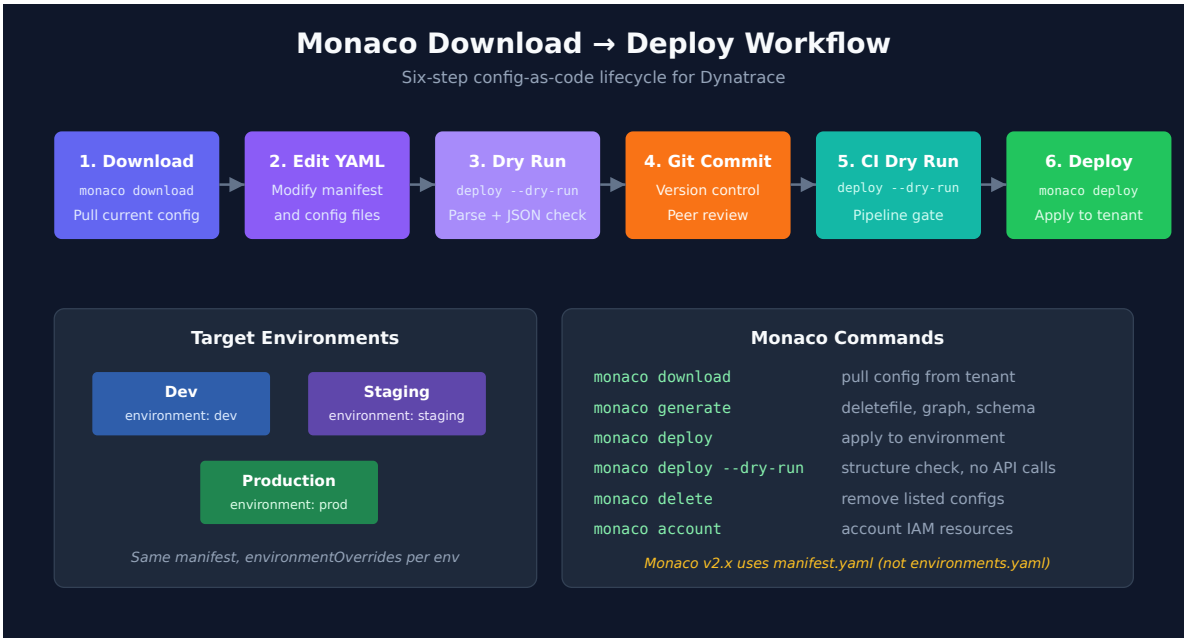
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --platform-token DT_PLATFORM_TOKEN \
  --output-folder ./backup/environment \
  --admin-access                          # Monaco CLI 2.30.0+, needs settings:objects:admin

monaco account download \
  --uuid "$DT_ACCOUNT_UUID" \
  --oauth-client-id DT_OAUTH_CLIENT_ID \
  --oauth-client-secret DT_OAUTH_CLIENT_SECRET \
  --output-folder ./backup/account        # account resources need an OAuth client
```

Commit both folders together with the gap list above. Checking the export against the tenant — object counts per config-type folder — is what turns "we ran download" into "we have a backup".

> **Sources:**
>
> - Monaco commands — Filtering of downloaded files (DT docs) — *"Typically, these types contain secrets that must not be exported"*; *"Read-only configurations are also excluded."*
> - Monaco YAML configuration — type fields (DT docs) — *"Monaco does not download Ready-made documents."*; *"existing commands like monaco deploy ignore any account configuration"*; Monaco settings client, v2.30.1 (Dynatrace GitHub) — *"AdminAccess requests resources of all owners for owner-based OpenPipeline schemas"*
> - Default settings download filters (Dynatrace GitHub) — *"builtin:host.monitoring.mode is not reliable during download"*
> - Monaco v2.30.0 release notes (Dynatrace GitHub) — *"Requires `settings:objects:admin`."*
> - Monaco v2.26.0 release notes (Dynatrace GitHub) — *"Account management: deploy, download, and delete boundaries"*
> - **Derived:** the "no Monaco type" row is the absence of those items from the type-fields page's type list, read 09/29/2026.

Download vs Deploy Workflow



---

6. Advanced Features

Configuration Dependencies

Reference another configuration with a **reference parameter** — `[project, configType, configId, property]`, or the long form with `type: reference`. The `configType` is the config type (`segment`, `slo-v2`, `workflow`, …) or, for Settings 2.0, the schema ID; `property: id` resolves to the referenced object's real Dynatrace ID at deploy time, and Monaco orders the deployment so the referenced object exists first. Here an SLO is scoped by a segment deployed from the same project:

```
configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: checkout-availability.json
      parameters:
        checkout_segment_id:
          type: reference
          configType: segment
          configId: checkout-production
          property: id
    type: slo-v2
```

In the template, scope the SLI with `"filterSegments": [{"id": "{{ .checkout_segment_id }}"}]` inside `customSli`. Because the segment ID is resolved per environment, the same config deploys to every tenant without a hard-coded ID — the portability problem S2S-07 spends a section on.

Classic equivalent: an alerting profile referencing a management zone — `management_zone_id: ["my-project", "builtin:management-zones", "production-mz", "id"]` on a `builtin:alerting.profile` config, used in the template as `{{ .management_zone_id }}`. Same mechanism, blocked schemas.

Conditional Configurations

`skip` is a boolean on `config:`. To deploy a config to only some environments, set a default and flip it per environment (or per environment group with `groupOverrides`):

```
configs:
  - id: production-only-config
    config:
      name: Production Detector
      template: detector.json
      skip: true
    type:
      settings:
        schema: builtin:davis.anomaly-detectors
        scope: environment
    environmentOverrides:
      - environment: production
        override:
          skip: false
```

Environment-Specific Overrides

`environmentOverrides` and `groupOverrides` sit at the config level, as siblings of `config:` — not inside it:

```
configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: checkout-availability.json
      parameters:
        slo_target: 99.0
    type: slo-v2
    environmentOverrides:
      - environment: production
        override:
          parameters:
            slo_target: 99.5
```

Grouping Configurations

Monaco v2 has no per-config `group:` key. Two different things are grouped, and neither is a config group:

- **Projects** group configurations. Put related configs in one project directory and deploy just that project with `--project`.
- **Environment groups** (`environmentGroups` in the manifest) group *target environments*. `--group` deploys to every environment in the named group; it does not select configs.

```
# Assumes a manifest with a "web-application" project and a "production" environment group
# (the §3 example has neither — there it would be --project my-project --group default)
monaco deploy manifest.yaml --project web-application --group production
```

Best Practices

Practice	Description
Use meaningful IDs	IDs should describe the config purpose
Modular structure	Separate configs by domain (alerting, monitoring, etc.)
Environment variables	Never hardcode tokens or URLs
Version control	Commit all changes with meaningful messages
Dry run first	Run `monaco deploy --dry-run` before applying — it catches YAML/JSON structure errors, not payload errors
Document parameters	Comment complex configurations
Treat a download as a partial backup	`monaco download` omits secrets, filtered and read-only settings, other users' objects and everything that is not configuration — keep the gap list from Using a Download as a Backup beside the export
Check upgrade status before you invest	Before building a long-lived project around a schema, confirm it is not marked **Blocked at upgrade** in AUTOM-02's catalog — a blocked schema takes the whole Monaco config type with it

Common Issues and Solutions

Issue	Cause	Solution
"Config already exists"	Duplicate ID	Use unique IDs or download first
"Schema not found"	Typo in schema ID — **or** the schema was removed when the tenant upgraded to the latest Dynatrace	Check the exact schema name in the API. If it was correct and worked previously, check the schema's upgrade status in AUTOM-02's catalog before assuming a typo
"Template not found"	Wrong path	Use relative path from config.yaml
"Validation failed"	Invalid JSON	Check JSON syntax in template

---

7. Next Steps

Monaco in a CI/CD Pipeline (the most common next step)

Monaco itself is a CLI — to make it part of a GitOps flow, wire it into your CI/CD platform of choice. **AUTOM-07: CI/CD Integration** has the recipe per platform:

Platform	AUTOM-07 section
GitHub Actions	§3 GitHub Actions

Platform	AUTOM-07 section
GitLab CI/CD	§4 GitLab CI/CD
Bitbucket Pipelines	§5.1 Bitbucket Pipelines — Monaco Deploy
Atlassian Bamboo	§6 Atlassian Bamboo — adapt the Plan Specs YAML to call `monaco deploy`
Azure DevOps	§7 Azure DevOps Pipelines — adapt the Pipeline YAML to call `monaco deploy`

For the **full sequenced path** from zero to a working pipeline, see **AUTOM-01 §6 First-Time Setup Path — Path B (Monaco target)**.

When to Consider Adding Terraform

Scenario	Why Terraform wins
Cross-system orchestration (cloud + Dynatrace + Git in one apply)	Monaco is Dynatrace-only
State management + drift detection on critical resources	Monaco has no state file and no drift view; a redeploy writes the template over whatever was changed in the UI
Part of larger IaC stack	Existing Terraform workflows extend naturally to Dynatrace
Lifecycle protections (`prevent_destroy` , `ignore_changes`)	Monaco has no per-resource lifecycle policy

For the framing of when a Terraform shop should *also* adopt Monaco (the reverse direction), see **AUTOM-01 §5: When a Terraform Shop Should Add Monaco**.

Continue the Series

Next Notebook	Focus
AUTOM-04: Terraform Provider	Infrastructure-as-code approach to Dynatrace configuration
AUTOM-07: CI/CD Integration	GitOps patterns for both Monaco and Terraform
AUTOM-08: Migration Automation	Tenant-to-tenant migration including `monaco download`

Additional Resources

- Monaco commands reference (DT docs) — *"A dry-run doesn't connect to Dynatrace and can't validate the content of the JSON sent to Dynatrace."*
- Monaco configuration YAML reference (DT docs) — parameters (`environment` , `reference`), `skip` , `environmentOverrides` / `groupOverrides`
- Monaco manifest and resources (DT docs) — *"the name value is the name of the environment variable that holds the secret, not the secret itself."*
- Install Monaco (DT docs)
- Monaco Documentation
- Configuration-as-code samples (Dynatrace GitHub)
- Configuration Schema Reference

Summary

In this notebook, you learned:

- How to install and configure Monaco
- Project structure and manifest configuration
- Creating and deploying configurations via YAML
- Advanced features: reference parameters, `skip`, environment/group overrides, and project-scoped deploys

> **Key Takeaway:** Monaco is ideal for teams wanting GitOps-style configuration management. It provides the right balance of simplicity and power for most Dynatrace automation needs. For the from-zero-to-pipeline sequence, see **AUTOM-01 §6** Path B.

---

*Continue to **AUTOM-04: Terraform Provider** to learn infrastructure-as-code patterns, or jump to **AUTOM-07: CI/CD Integration** to wire Monaco into a pipeline now.*

Community Resources & Examples

The following GitHub repositories provide starter templates, real-world examples, and hands-on exercises for Monaco:

Official Repositories

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.30.0 at time of writing, released 09/23/2026 — check releases for newer) -- Go binary, Apache-2.0
community-examples / configuration-as-code	Current home of the Monaco samples, including the 9 starter templates in `basic-templates-monaco`. The old `dynatrace-configuration-as-code-samples` repo is archived: *"The Dynatrace Configuration as Code samples are no longer maintained here."*
easytrade	Demo microservices app with a working `monaco/` directory (manifest.yaml, detection rules, workflows)
Dynatrace-Config-Manager	**Archived** — *"This repository has been archived and is no longer maintained."* Historical reference only; use `monaco download` for tenant-to-tenant migration (AUTOM-08)

Starter Templates (in `community-examples/configuration-as-code`)

Template Directory	What It Configures
`basic-templates-monaco`	Alerting, app detection, synthetic, maintenance window, management zones, ownership, notifications, SLOs — 8 of its 13 configs use schemas on the removed in Latest Dynatrace (DT docs) list (management zone, alerting profile, problem notification, maintenance window, classic SLO); use them as Monaco syntax examples, not as templates for a latest-Dynatrace tenant

Template Directory	What It Configures
`learn-monaco-auto-tag`	Auto-tagging with Monaco
`account-monaco-admin-access`	Admin access setup for Monaco

Pipeline Observability Samples

Monaco configurations for ingesting CI/CD pipeline events via OpenPipeline:

Directory	CI/CD Platform
`github_pipeline_observability`	GitHub Actions
`gitlab_pipeline_observability`	GitLab CI
`azure_devops_observability`	Azure DevOps
`argocd_observability`	ArgoCD

Training & Exercises

Repository	Description
monaco-self-paced-exercises	6 structured exercises: install, auto-tag, download, variables, delete/restore, linking configs
monaco-demo	Working GitHub Actions workflow for Monaco deploy with "crawl-walk-run" adoption methodology

> **Note:** Monaco v1 (`dynatrace-oss/dynatrace-monitoring-as-code`) is no longer supported. Current Monaco releases have no conversion command: *"The last Monaco version to support the convert command is Monaco version 2.19.0."* Either run Monaco 2.19.0's `convert` per the migrating to v2 guide (DT docs), or re-run `monaco download` against the source tenant.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.